FFT, KMeans y PCA para señales

Pablo Cobo, Pablo López, Dong Hyun An, José María García

Matemáticas para el Aprendizaje Automático

20 de abril de 2026

Índice

1. Datos escogidos e interpretación
2. DFT
3. FFT
4. K-means
5. PCA
6. Bibliografía

Datos escogidos e interpretación

```
# 1. Generar Datos
np.random.seed(42)
n_points = 500
t = np.linspace(0, 1, n_points)#Segundos

walking = np.array([np.sin(2 * np.pi * np.random.uniform(1.5, 2.5) * t) + np.random.normal(0, 1, n_points)
  for _ in range(100)])
running = np.array([np.sin(2 * np.pi * np.random.uniform(3, 4) * t) + np.random.normal(0, 1, n_points) for
  _ in range(100)])
X_time = np.vstack((walking, running))
print(len(X_time),len(X_time[0]))
```

La DFT (Discrete Fourier Transform)

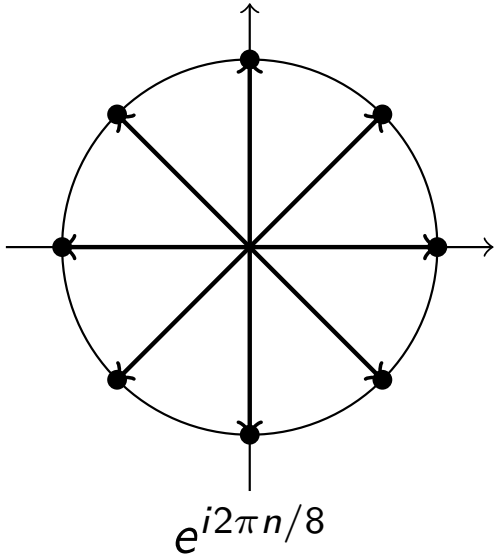
La DFT es una transformación que descompone una secuencia de datos discretos en las frecuencias que contienen esos datos.

- La fórmula que usa es:

$$\hat{x}_k = \sum_{n=0}^{N-1} x_n \, e^{-i2\pi kn/N}, \quad k = 0, \ldots, N-1$$

- Cada $\hat{x}_k$ corresponde a la frecuencia k
- $e^{-i2\pi n/N}$ son las **raíces N-ésimas de la unidad**

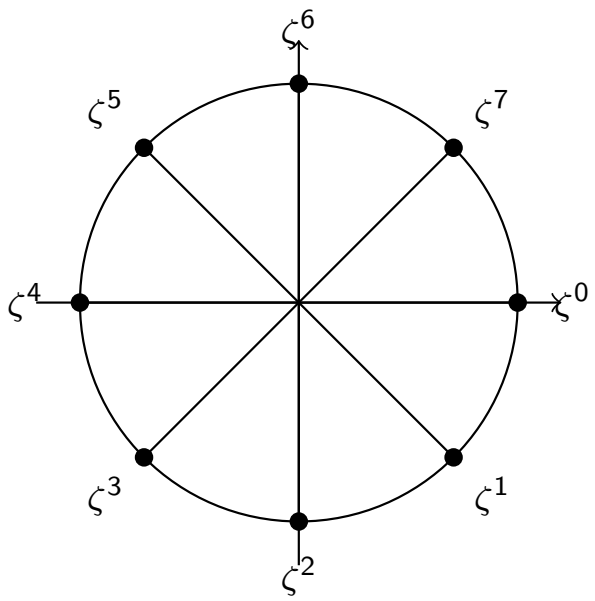
Raíces de la unidad ($N = 8$)



¿Por qué funciona?

$$\hat{x}_0 = x_0\zeta^{0\cdot 0} + x_1\zeta^{1\cdot 0} + x_2\zeta^{2\cdot 0} + x_3\zeta^{3\cdot 0} + x_4\zeta^{4\cdot 0} + x_5\zeta^{5\cdot 0} + \dots$$
$$\hat{x}_1 = x_0\zeta^{0\cdot 1} + x_1\zeta^{1\cdot 1} + x_2\zeta^{2\cdot 1} + x_2\zeta^{3\cdot 1} + x_4\zeta^{4\cdot 1} + x_5\zeta^{5\cdot 1} + \dots$$
$$\hat{x}_2 = x_0\zeta^{0\cdot 2} + x_1\zeta^{1\cdot 2} + x_2\zeta^{2\cdot 2} + x_3\zeta^{3\cdot 2} + x_4\zeta^{4\cdot 2} + x_5\zeta^{5\cdot 2} + \dots$$
$$\zeta = e^{-i2\pi/N}$$

- ζ^n siempre es una raíz N-ésima de la unidad $\zeta^n = \zeta^{n(modN)}$.
- Lo que cambia de un k a otro es qué raíz multiplicamos por cada x_n
- Podemos ver k como la velocidad a la recorremos el círculo unidad.

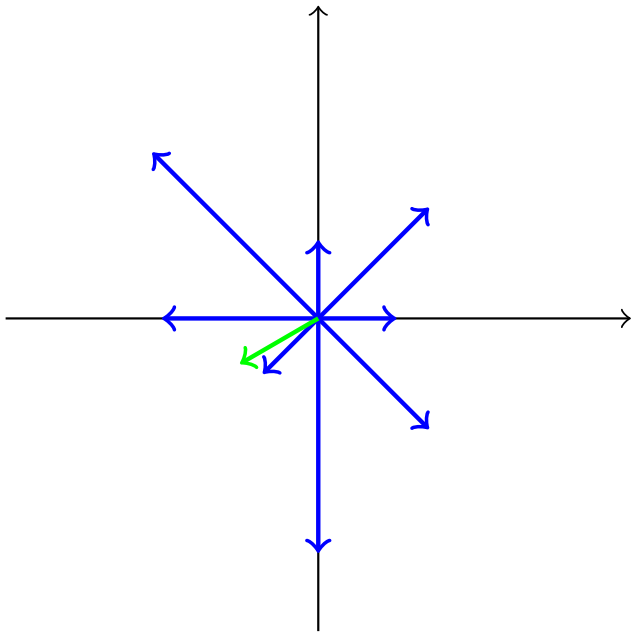


¿Por qué fuciona?

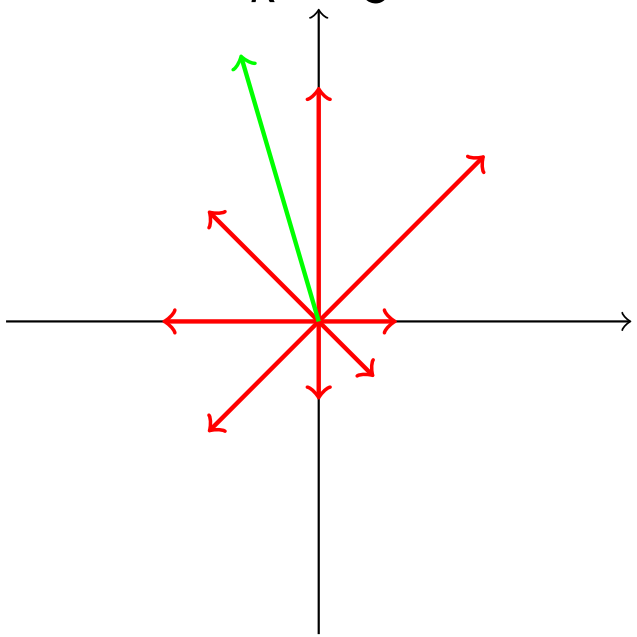
Señal:

$$x = [0{,}5\ 1\ 1{,}5\ 0{,}5\ 1\ 1{,}5\ 0{,}5\ 1]$$

$k = 1$



$k = 3$



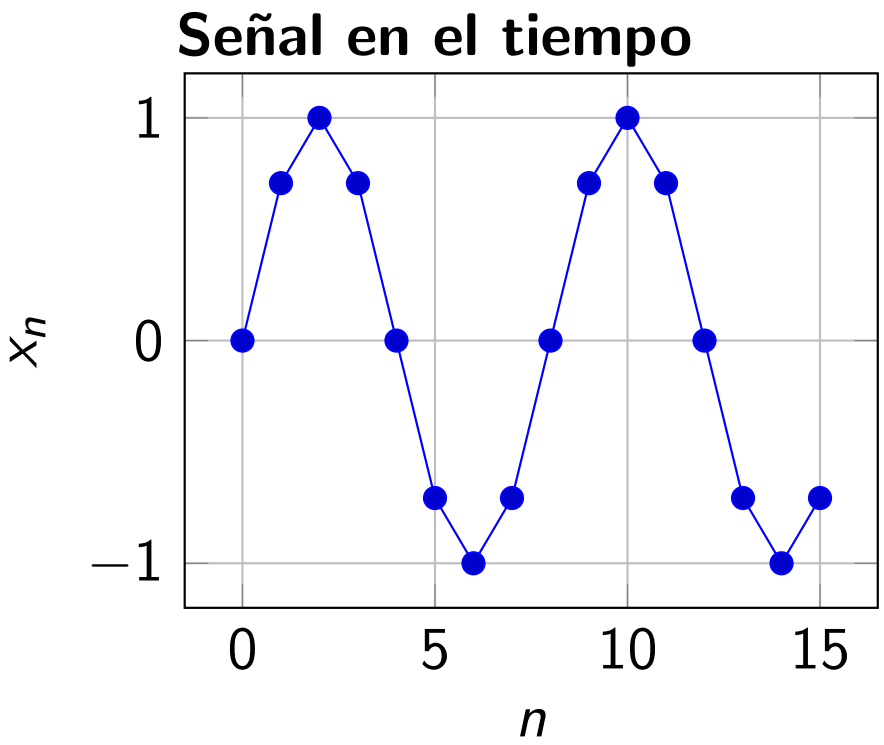
Interpretación:

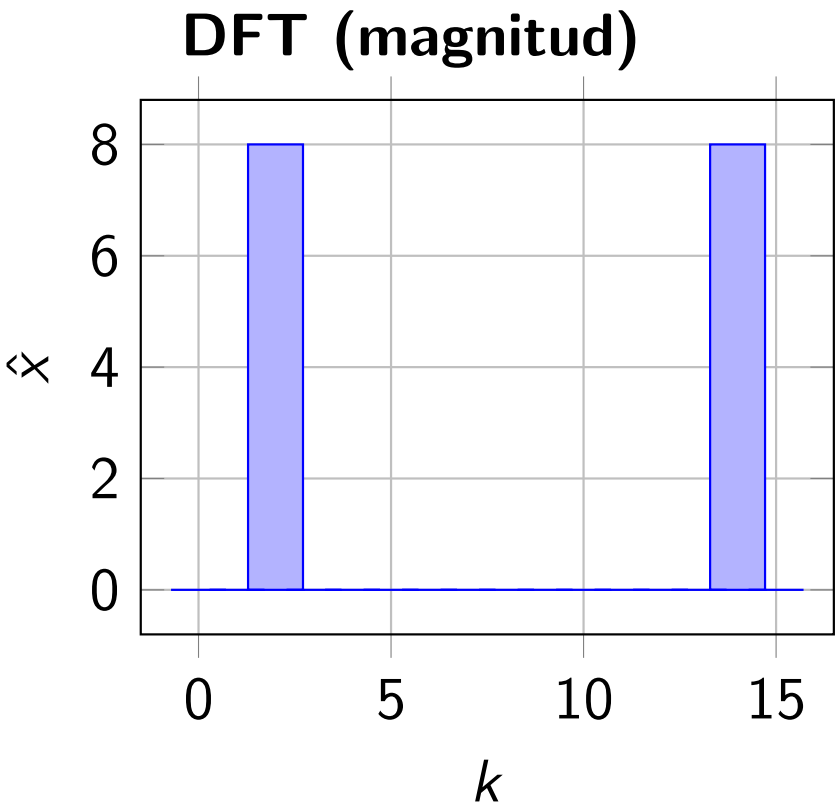
- Cada vector tiene módulo x_n
- El ángulo depende de k
- Diferentes $k \to$ distinta frercuencia $\to$ distinta suma

Ejemplo y gráficas

Señal:

$$x_n = \sin\left(\frac{4\pi}{16}n\right)$$





- Se toma el módulo del número complejo $\hat{x}$
- La DFT muestra picos en $k = 2$ y $k = 14$ (simetría conjugada)

Forma matricial de la DFT y complejidad

La DFT en forma matricial se puede escribir explícitamente como:

$$\begin{bmatrix} \hat{x}_0 \\ \hat{x}_1 \\ \vdots \\ \hat{x}_{N-1} \end{bmatrix} = \begin{bmatrix} F_{0,0} & F_{0,1} & \cdots & F_{0,N-1} \\ F_{1,0} & F_{1,1} & \cdots & F_{1,N-1} \\ \vdots & \vdots & \ddots & \vdots \\ F_{N-1,0} & F_{N-1,1} & \cdots & F_{N-1,N-1} \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ x_{N-1} \end{bmatrix}$$

Elementos de la matriz:

$$F_{k,n} = e^{-i2\pi kn/N}$$

Observación: Complejidad computacional es $O(N^2)$. Queremos un algoritmo más eficiente: La Transformada Rápida de Fourier (FFT)

FFT

- 'Es el algoritmo númerico más importante de nuestra época' - Gilbert Strang.
- Reduce la complejidad de $O(N^2)$ a $O(N \log N)$.
- Usos principales:
  - Señales y procesamiento de datos.
  - Resolución de PDEs numéricos.
  - Compresión de audio e imagen.
  - Multiplicación rápida de polinomios y enteros.
  - Convoluciones rápidas y aplicaciones en Machine Learning.

Algoritmo FFT (Cooley-Tukey)

Suponemos que N es potencia de 2.

1. Partimos de la definición de la DFT:

$$\hat{x}_k = \sum_{n=0}^{N-1} x_n \, e^{-i2\pi kn/N}, \quad k = 0, \ldots, N-1$$

2. Dividimos la suma en índices pares e impares:

$$\hat{x}_k = \sum_{m=0}^{N/2-1} x_{2m} \, e^{-i2\pi k(2m)/N} + \sum_{m=0}^{N/2-1} x_{2m+1} \, e^{-i2\pi k(2m+1)/N}$$

3. Sacamos el factor común de los índices impares:

$$\hat{x}_k = \sum_{m=0}^{N/2-1} x_{2m} \, e^{-i2\pi km/(N/2)} + C_k \sum_{m=0}^{N/2-1} x_{2m+1} \, e^{-i2\pi km/(N/2)},$$

donde $C_k = e^{-2\pi ik/N}$

Algoritmo FFT (Cooley-Tukey)

5. Como ahora los exponentes están divididos por $N/2$, los valores se **repiten** para $k \geq N/2$. Por eso, basta calcular las sumas solo para

$$0 \leq k \leq N/2 - 1.$$

   Además, la relación entre los factores exponenciales es:

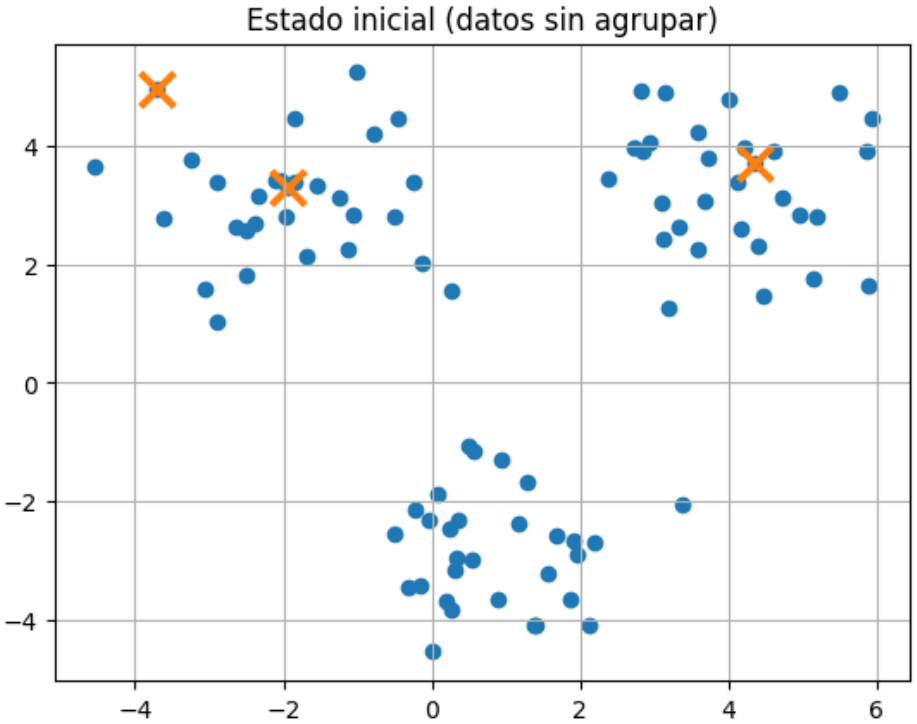
$$C_{k+N/2} = -C_k = -e^{-2\pi ik/N}.$$

6. **Recursión:** Aplicamos el mismo procedimiento a cada sub-señal hasta que cada sub-señal tenga tamaño 1. Luego vamos combinando los resultados (sumando los pares e impares con el factor C_k) para obtener $\hat{x}_k$ final.

K-means

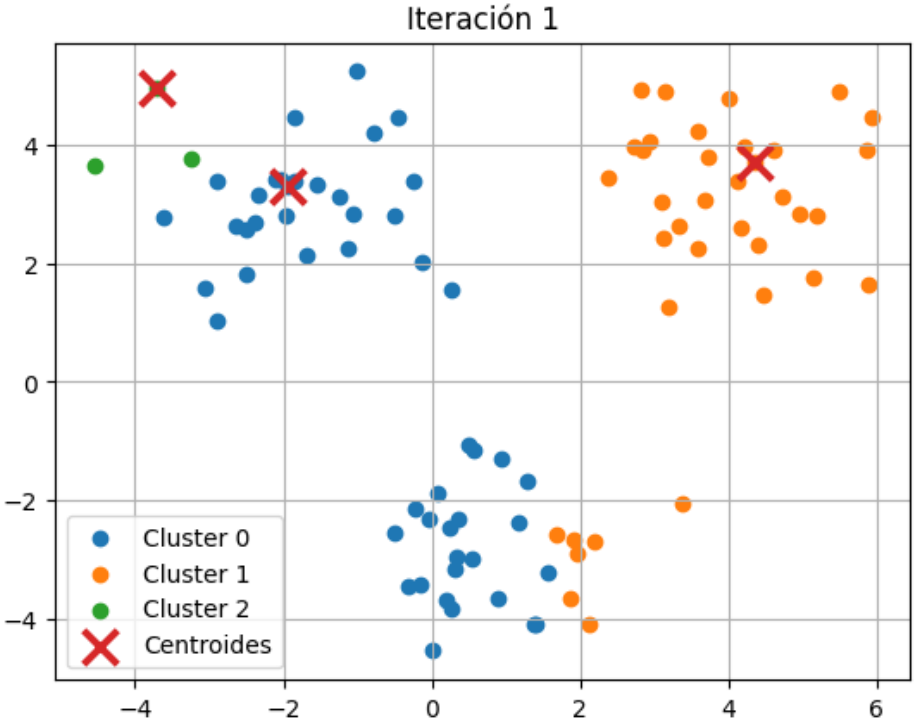
- K-means es un algoritmo de clustering no supervisado.
- **Objetivo**: agrupar los datos en K clusters.

1. Elección al azar de tres puntos como centroides



Cálculo de distancias y primer clustering

2. Cálculo de distancias (euclídeas) de los puntos a los centroides
3. Reunir en clusters los puntos según qué centroide está a menor distancia

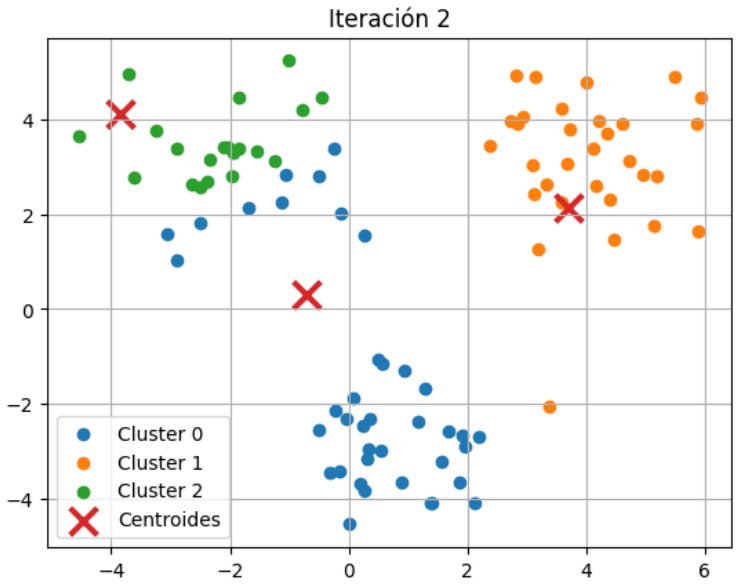


Cálculo de nuevos centroides

4. Calcular la posición media de cada cluster:

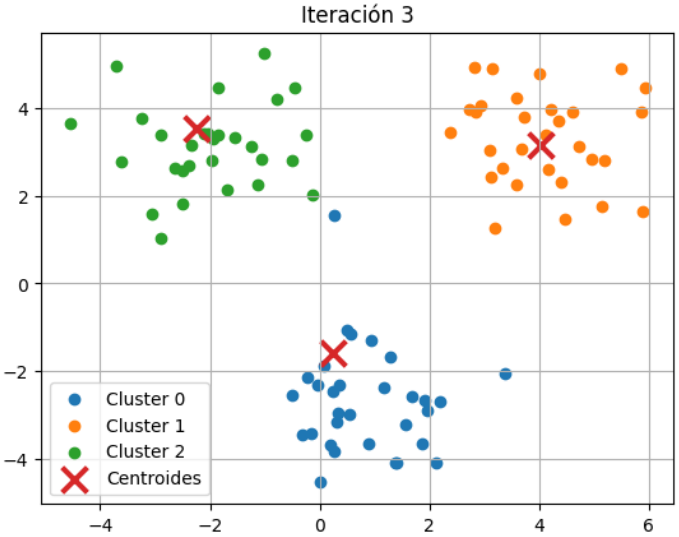
$$C_j = \frac{1}{n_j} \sum_{i=1}^{n_j} \mathbf{x}_i$$

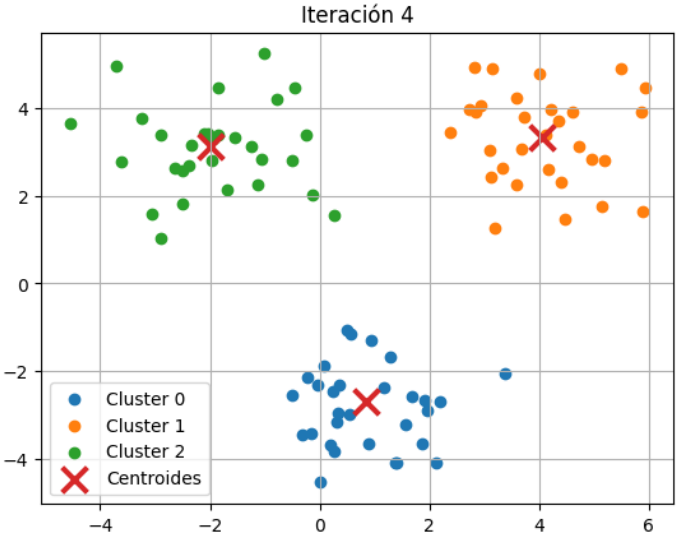
   Con n_j el numero de elementos de cada cluster.
5. Asignar cada posición media como nuevo centroide
6. Repetir los pasos anteriores con los nuevos centroides

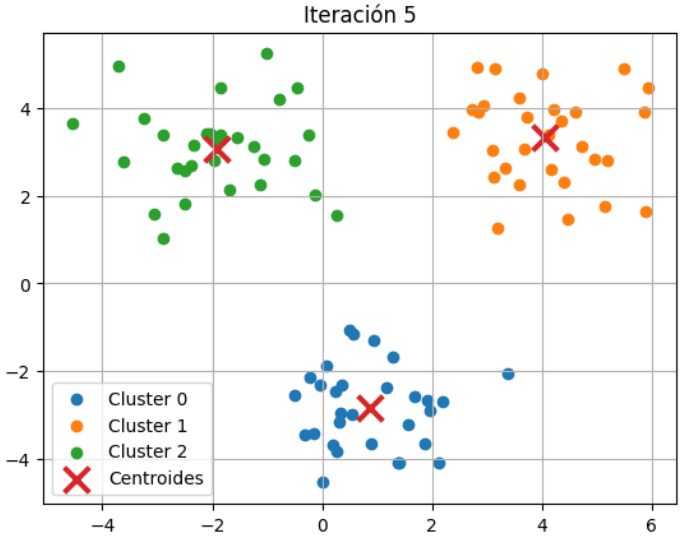


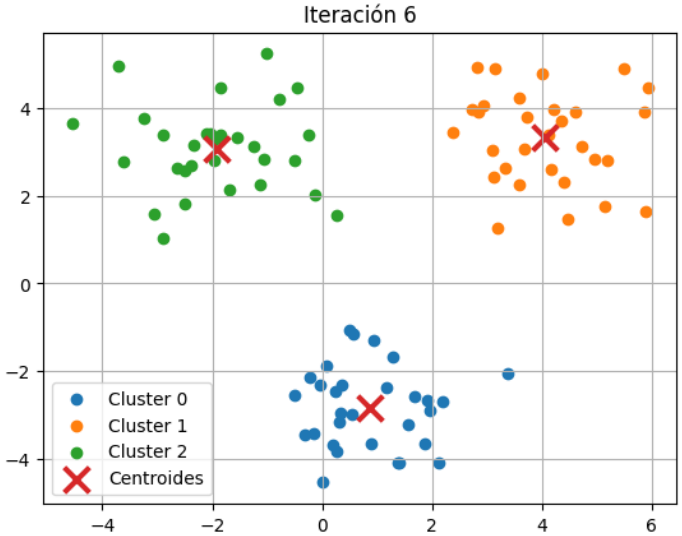
Iteraciones

7. Parar cuando los centroides no cambian de una iteración a la siguiente.









¿Qué K tomar?: Método del codo

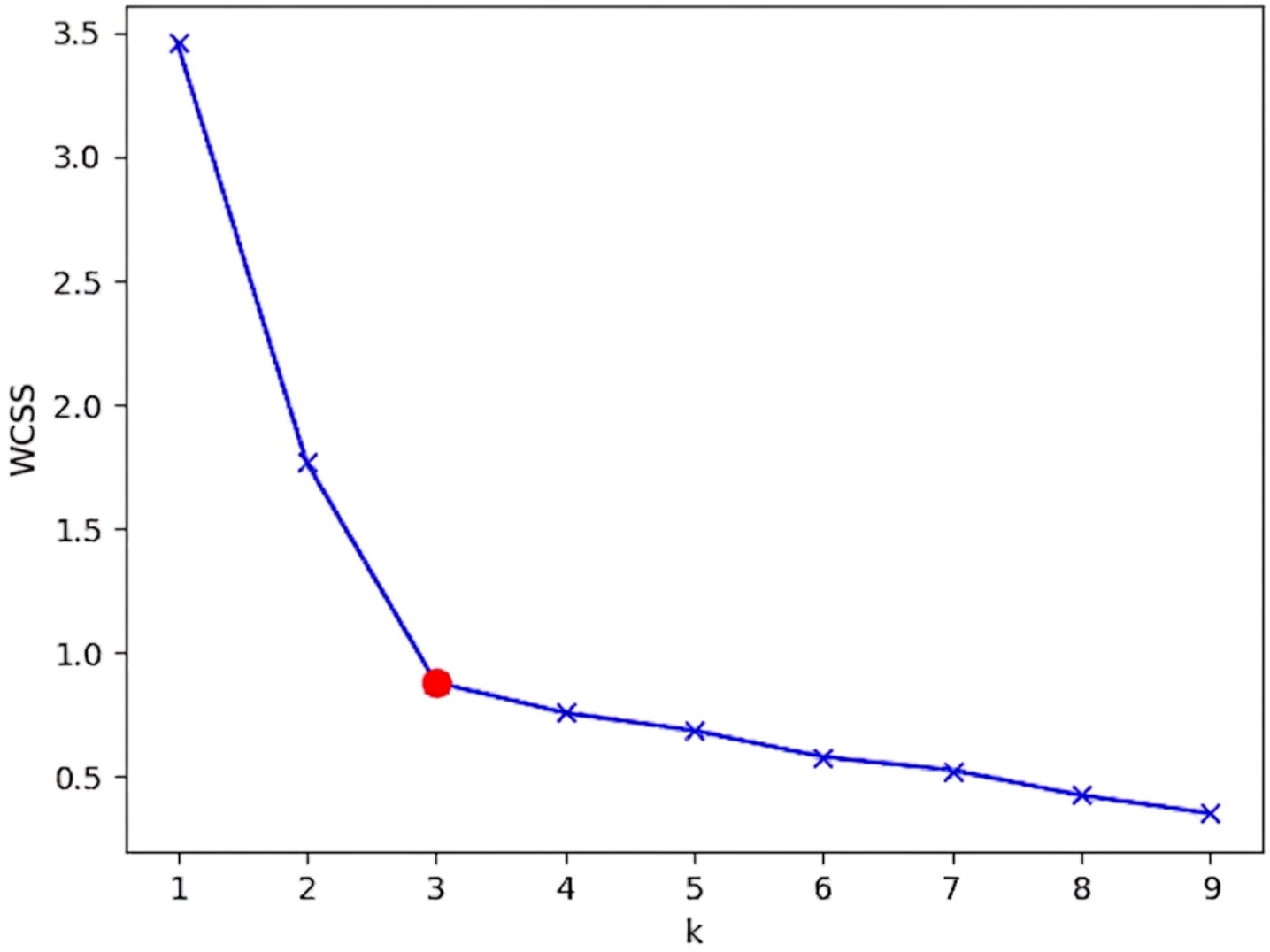
- El método del codo sirve para ayudar a elegir el número de clusters en los que dividir los datos.
- Procedimiento:

1. Hacer K-means para distintos K
2. Calcular para cada K el WCSS (Within-Cluster Sum of Squares):

$$\text{WCSS} = \sum_{k=1}^{K} \sum_{x_i \in C_k} \|x_i - \mu_k\|^2$$

3. Graficar estos valores
4. Quedarnos con el K donde el WCSS de K+1 no sea "mucho menor"que el de K. En la gráfica es el K que ejerce de "*codo*".

Ejemplo Método del codo



K-means para los datos

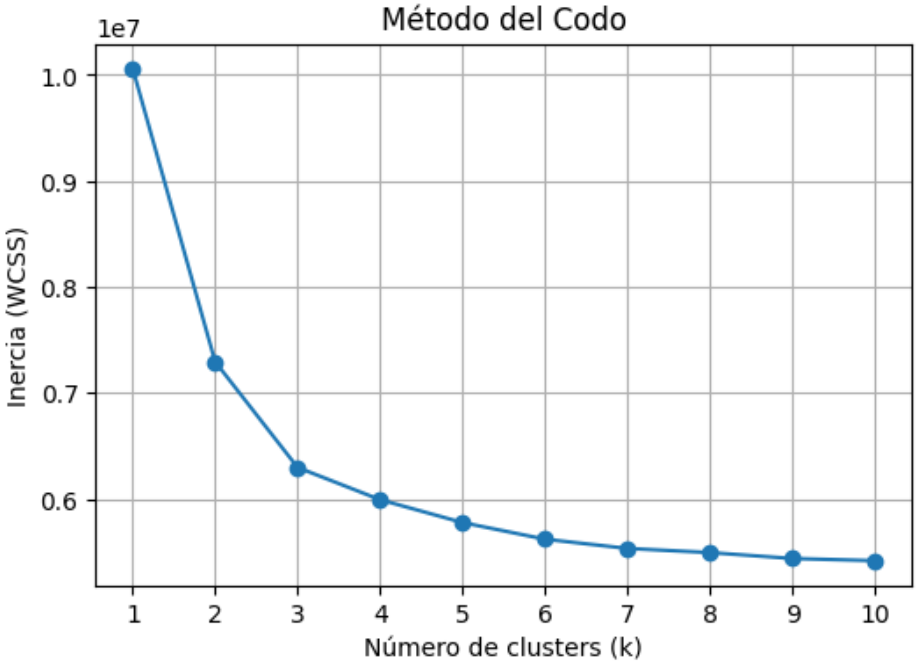
```python
# 3. K-means SIN StandardScaler
# IMPORTANTE: No escalamos columna por columna para no amplificar el ruido de las frecuencias vacías
kmeans = KMeans(n_clusters=2, random_state=42, n_init=100)
clusters = kmeans.fit_predict(X_mag)
```

- *n_clusters* $\rightarrow$ número de clusters
- *random_state* $\rightarrow$ siempre se empieza por los mismos centroides
- *n_init* $\rightarrow$ número de inicializaciones con nuevos primeros centroides

Se queda de los 100 pares de clusters con el par con menor WCSS

Método del codo para los datos

Al hacer el método del codo a los datos queda la siguiente gráfica:



Parece que la elección óptima de K es 3.

Resultados de K-means para los datos

Separa perfectamente para K=2, y divide el grupo de running para K=3.

Distribución con K=2
(Separación Perfecta)

	Running	Walking
Cluster 0	0	100
Cluster 1	100	0

Distribución con K=3
(Sobresegmentación)

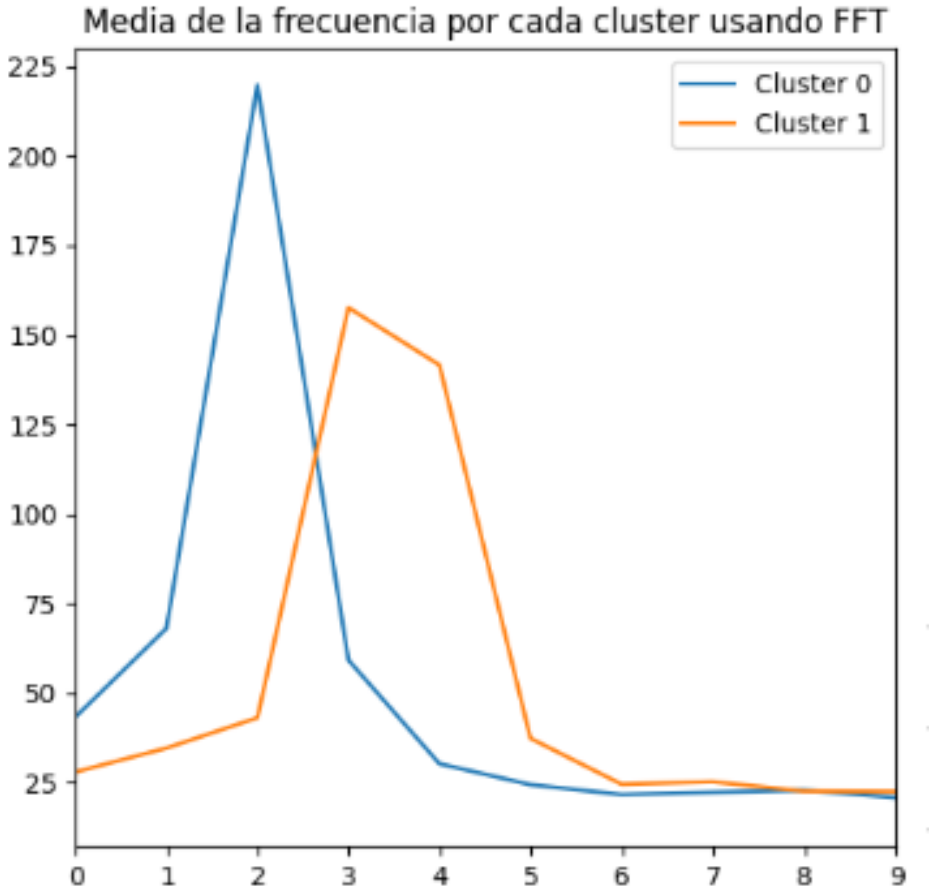
	Running	Walking
Cluster 0	41	0
Cluster 1	0	100
Cluster 2	59	0

```
Varianza walking: 129.13780997177673
Varianza running: 161.447618014789
```

Nuestra implementación en el código

```python
# 2. Transformada de Fourier
X_mag = np.abs(np.fft.rfft(X_time))
freqs =np.fft.rfftfreq(n_points, d=t[1]-t[0])

# 3. K-means
X_input = X_mag

kmeans = KMeans(n_clusters=2, random_state=42, n_init=100)
clusters = kmeans.fit_predict(X_input)

# 4. Visualización
plt.figure(figsize=(15, 5))

# Plot A: Espectro promedio
plt.subplot(1, 3, 1)
for cluster_id in range(2):
    cluster_mean = X_mag[clusters == cluster_id].mean(axis=0)
    plt.plot(freqs, cluster_mean, label=f'Cluster {cluster_id}')
plt.title("Media de la frecuencia por cada cluster usando FFT")
plt.xlim(0,9)
plt.legend()
```

Nuestra implementación en el código



Problema de dimensionalidad

Nuestra Matriz de datos procesados tras aplicar `np.fft.rfft`:

$$\texttt{X_mag} \in \mathbb{R}^{200\times 251}$$

Problemas:

- Imposible de visualizar.
- Muchas coordenadas correlacionadas.
- Información redundante.

Solución: Método para hallar componentes principales y **reducir** así la dimesión de los datos con la **pérdida mínima** de información (PCA, método del gradiente descendente, autovectores de la matriz de covarianzas ...)

PCA

Al contrario que el método del gradiente descendente, que es iterativo, y lento en casos como este, donde el tamaño de los datos es "grande", el *Principal Component Analysis* (**PCA**) es un método **directo** y **estable** para casos como el de `X_mag`. **Funcionamiento** (Algoritmo PCA(**n**) de `scikitkearn.decomposition`):

1. **Centrar** los datos:

```
X_centered = X_input - X_input.mean(axis=0)
```

2. Aplicar **SVD**:

$$\texttt{U,S,V}^t = \texttt{np.linalg.svd(X_centered)}$$

3. Escoger **cúantas** componentes principales queremos:

Escoge las **n primeras filas** de $\texttt{V}^t$

El problema de la elección de las componentes

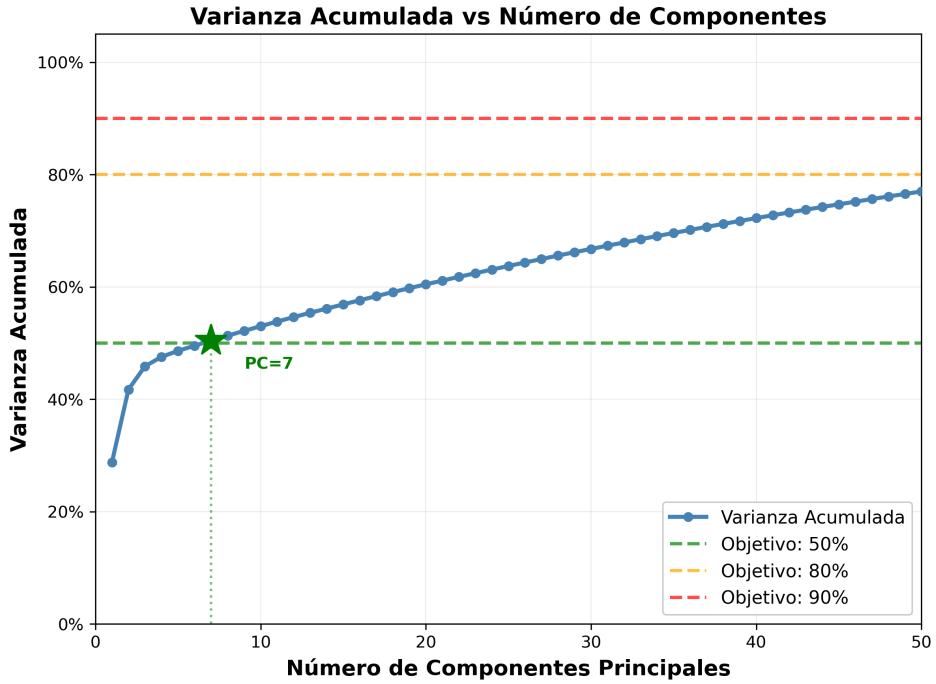
Queremos preservar la **máxima** información de los datos, es decir, queremos preservar la **máxima varianza acumulada** por las componentes principales. Pero a la vez también queremos poder **visualizar** los datos.

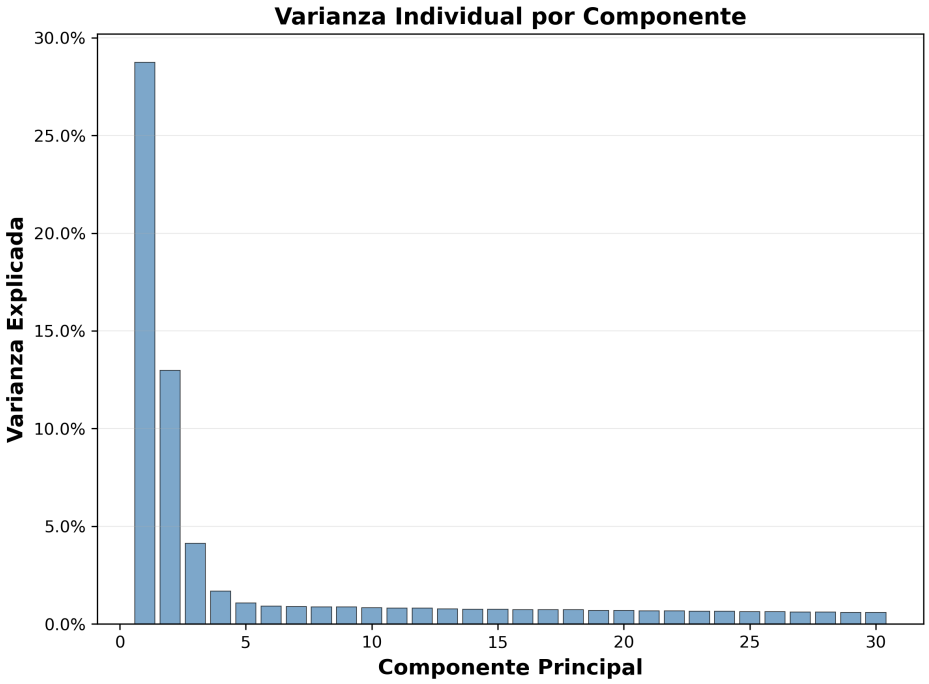
En nuestro caso, **entrenamos** con `X_mag` la PCA, utilizando: `PCA().fit(X_mag)`, que nos devolverá, entre otros outputs, el **ratio de variancia**, vector en $\mathbb{R}^{200}$, que contiene la varianza de **cada una** de las componentes principales.

Tras esto, es fácil ver que es **imposible**, en este caso, visualizar nuestros datos sin perder más de la mitad de la información.

El problema de la elección de las componentes

Hemos elegido **n=2**. Estos gráficos muestran la **suma acumulativa** de los elementos del ratio de variancia y el propio **vector** ratio de variancia respectivamente.





Implementación en el código e interpretación

PC1 podría representar la **velocidad**, y PC2 el **cómo afecta el ruido** a cada tipo de muestra.

```
#Plot B: PCA
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_mag)

plt.subplot(1, 3, 2)

cmap = plt.cm.coolwarm
n_clusters = len(np.unique(clusters))

for cluster_id in range(n_clusters):
    plt.scatter(
        X_pca[clusters == cluster_id, 0],
        X_pca[clusters == cluster_id, 1],
        label=f'Cluster {cluster_id}',
        color=cmap(cluster_id / (n_clusters - 1)),
        alpha=0.7
    )

plt.title("PCA")
plt.legend()
```

Bibliografía

Cooley, J. W., Tukey, J. W. (1965). An algorithm for the machine calculation of complex Fourier series. *Mathematics of Computation*, 19(90), 297-301.

Jolliffe, I. T. (2002).*Principal Component Analysis* (2nd ed.). Springer-Verlag.

MacQueen, J. (1967). Some methods for classification and analysis of multivariate observations. In *Proceedings of the Fifth Berkeley Symposium*, 281-297.

Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.

Hastie, T., Tibshirani, R., Friedman, J. (2009). *The Elements of Statistical Learning* (2nd ed.). Springer-Verlag.

Gracias por su atención

¿Preguntas?